Tallinn University of Technology

3D Simulation (SD)

Author: Jordan Thomas Brodie
Student Code: 243483
Individual variant: RPR8
Course: ATR0030 Robotics

October 19, 2026

1 Task

The purpose of this exercise was to build and simulate my assigned RPR8 manipulator in MATLAB Simulink using Simscape Multibody. Unlike the earlier forward kinematics (FK) and workspace (WS) exercises, this model uses physical bodies and joints to show the manipulator moving in three dimensions. The assignment also provided a way to compare the position of the end effector (EE) in the simulation with the position calculated from the FK model.

I made the RPR8 model with three joints (revolute–prismatic–revolute), added solid links with their mass properties, provided motion inputs, and measured the EE position relative to the world frame. The simulation was run from $t = 0$ to $t = 10$ s, and a video was exported separately for submission.

2 Initial Data

My assigned manipulator is the RPR8 variant. The geometry is the same as in the previous FK and WS exercises. Table 1 shows the dimensions used in the simulation.

Table 1: Initial link dimensions for variant RPR8

Parameter	Value	Description
L_0	3 m	Base offset
L_1	5 m	J1–J2 link length
$L_{2,\max}$	6 m	Maximum prismatic extension
L_3	5 m	J3–EE link length

The home position is defined by $A = 0$, $B = 0$, and $L_2 = 6$ m. The fixed transform T_{01} offsets J1 by L_0 along $+z$, R_{01} rotates the frame by $-\pi/2$ about y, and T_{12} and T_{3E} provide the remaining fixed link lengths. The prismatic displacement from J2 to J3 is in the local $-z$ direction, matching my FK model.

3 Simscape Multibody Model

The complete block diagram is shown in Figure 1. The mechanical chain begins at the World Frame and passes through the rigid transforms $T01$ and $R01$, the revolute joint J1, rigid transform $T12$, the prismatic joint J2, revolute joint J3, and the final rigid transform $T3E$ to the EE.

The solid links were added as Brick Solid bodies so they would be visible in Multibody Explorer. Their inertia was calculated from the geometry using a positive density. The EE was marked with a light-purple sphere of radius 0.3 m. A Mechanism Configuration block and Solver Configuration block were included in the model.

Each joint was set to *Motion: Provided by Input* and *Force/Torque: Automatically Computed*. Simulink–PS Converters supplied the requested motions in radians for J1 and J3, and meters for J2. The J1 converter used second-order input filtering so that the required motion derivatives were available to the solver.

To measure the EE location, I connected a Transform Sensor between the World Frame and the frame at $T3E$. The sensor's XYZ translation output passed through a PS–Simulink Converter into a Scope, producing a plot of the EE coordinates over the simulation time.

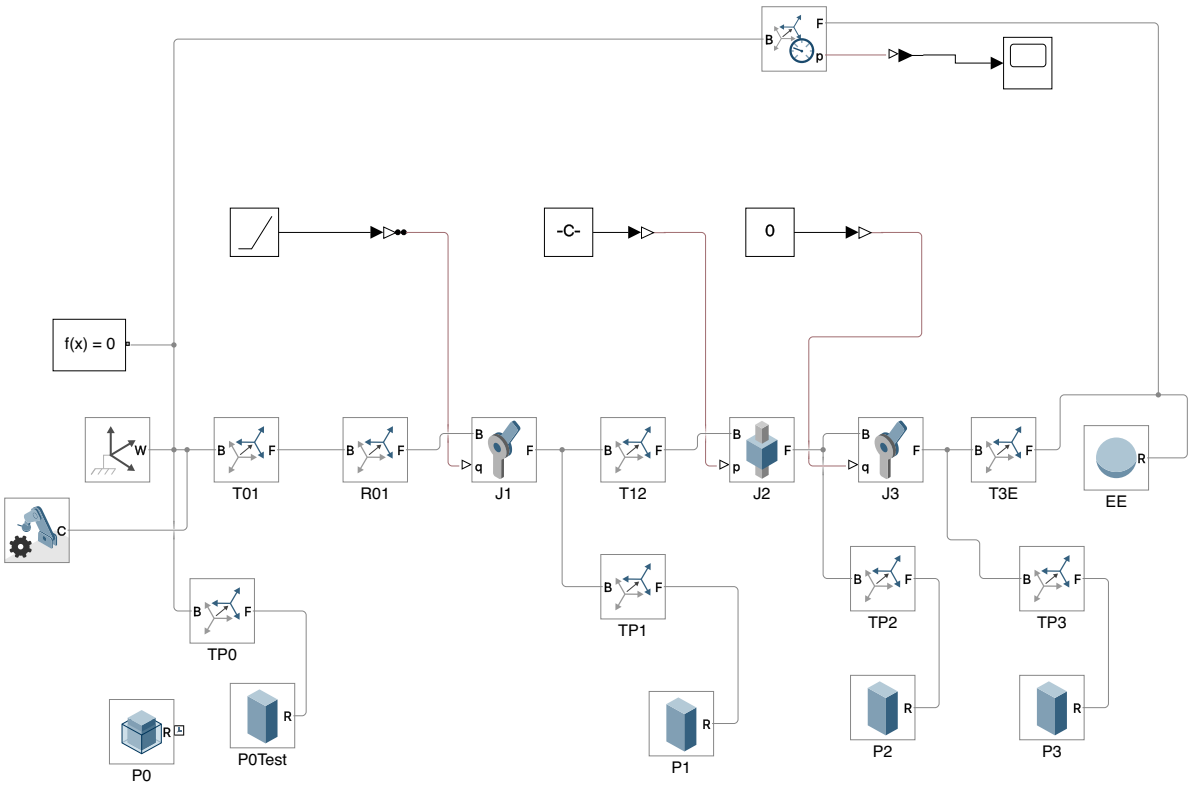


Figure 1: Complete RPR8 model in Simulink/Simscape Multibody, including motion inputs, solid links, the EE, and the Transform Sensor.

4 Motion Inputs and Simulation

For this test I moved only the first revolute joint while holding the prismatic extension and the last revolute joint fixed. The commanded motion was

$$A_{\mathrm{cmd}}(t) = \frac{\pi}{40}\,t, \qquad p_2(t) = -6 \text{ m}, \qquad B(t) = 0, \qquad 0 \le t \le 10 \text{ s}. \tag{1}$$

The first joint therefore had a commanded final angle of $A_{\mathrm{cmd}}(10) = \pi/4 = 45°$. The prismatic joint stayed at its extended home position and J3 stayed at zero degrees.

Figure 2 shows the home configuration at $t = 0$ s. Figure 3 shows the final configuration at $t = 10$ s. The links move together as J1 rotates and the purple EE marker follows the end of the last link. Both views use the same camera orientation so the change in the configuration is easy to compare.

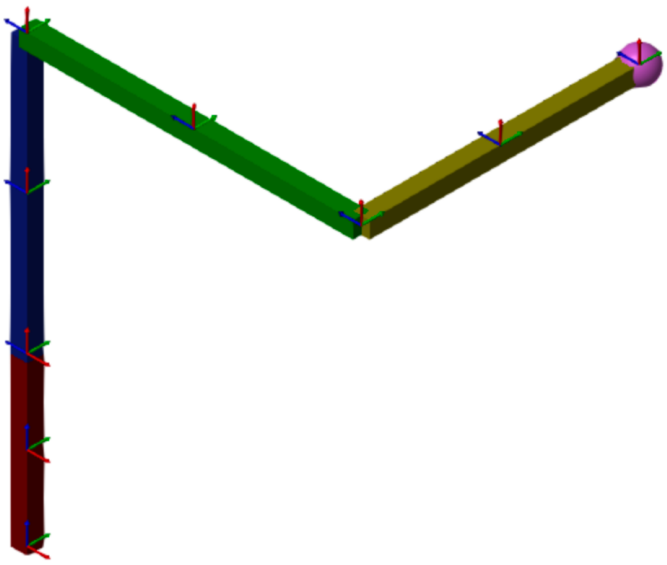


Figure 2: Initial configuration of the RPR8 manipulator at $t = 0$ s, with the end effector marked by the purple sphere.

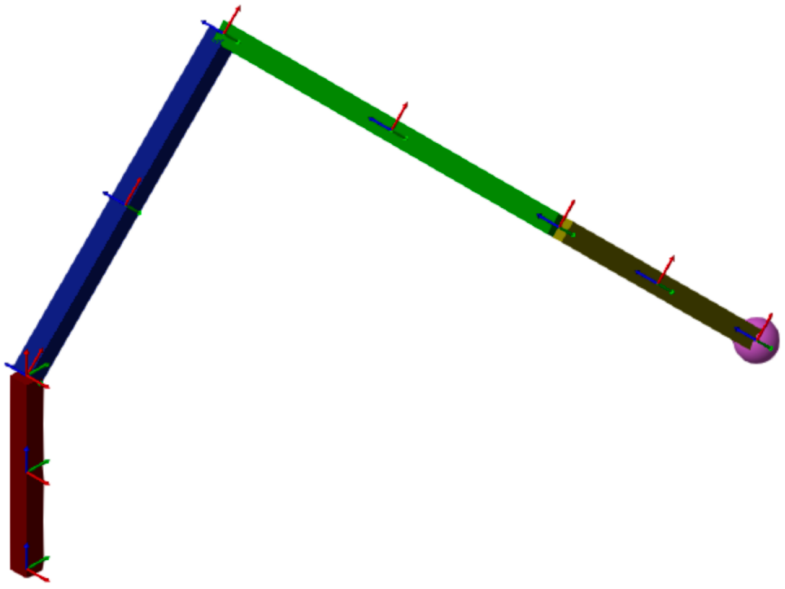


Figure 3: Final configuration of the RPR8 manipulator at $t = 10$ s after the commanded J1 rotation.

5 End-Effector Position and FK Verification

For $B = 0$ and the fixed prismatic extension $L_2 = 6$ m, the homogeneous transformation sequence from the previous FK exercise can be written as

$$ {}^{0}T_{E} = T_{01}\, R_{01}\, R_z(A)\, T_{12}\, T_{23}\, R_z(B)\, T_{3E}. \tag{2} $$

With the link values from Table 1, this gives the EE position in the world frame as

$$ \mathbf{p}_{EE}(A) = \begin{bmatrix} 6 \\ 5\sin A + 5\cos A \\ 3 + 5\cos A - 5\sin A \end{bmatrix} \text{ m}. \tag{3} $$

At $A = 0$ the theoretical EE position is $[6,\ 5,\ 8]^T$ m. At the commanded angle $A = \pi/4$ it is $[6,\ 7.0711,\ 3.0000]^T$ m.

The actual coordinates were measured with the Transform Sensor and read from the Scope cursors at $t = 0$ and $t = 10$ s. Table 2 compares the FK calculation with the simulation.

Table 2: Comparison of theoretical and measured end-effector coordinates

Time	Coordinate	FK (m)	Simscape (m)	$\lvert\Delta\rvert$ (m)
0 s	x	6.0000	6.0000	0.0000
0 s	y	5.0000	5.0000	0.0000
0 s	z	8.0000	8.0000	0.0000
10 s	x	6.0000	6.0000	0.0000
10 s	y	7.0711	7.0711	0.0000
10 s	z	3.0000	3.0111	0.0111

Figure 4 shows the EE position during the complete ten-second test. The x coordinate stays at 6 m, the y coordinate increases from 5 m to approximately 7.0711 m, and the z coordinate decreases from 8 m to 3.0111 m. This agrees closely with the expected motion in the yz plane.

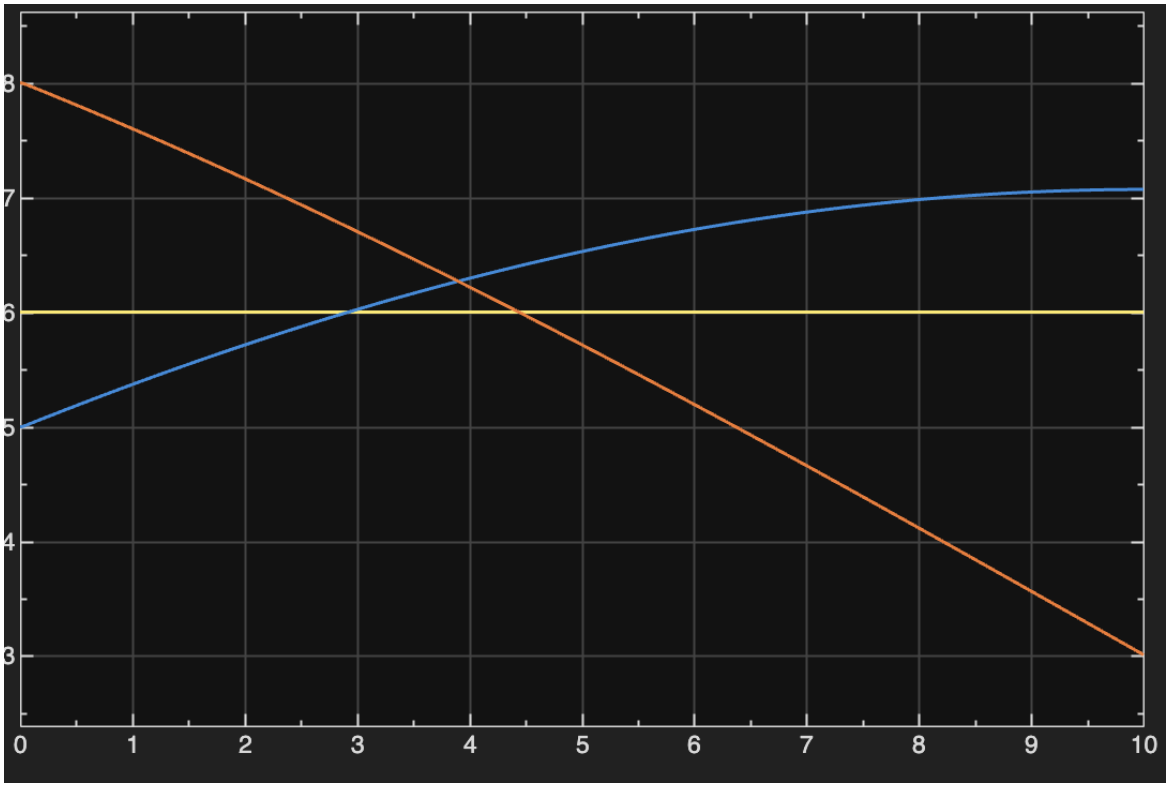


Figure 4: Transform Sensor Scope output for the EE position: x (yellow, constant), y (blue, increasing), and z (orange, decreasing) over 0–10 s.

The final z-coordinate differs from the theoretical value by 0.0111 m (about 1.11 cm). This small difference is consistent with the input filtering used for the motion signal: at the end of the simulation, the filtered joint angle can lag slightly behind the ideal ramp command. The measured result is nevertheless very close to the FK prediction.

6 Conclusion

I thought this assignment was really neat because it showed me another way to use MATLAB. I had already used MATLAB in my electronics class to simulate circuits, but I did not realize how much you could do with moving bodies in Simscape Multibody. It was interesting to see that you can build the geometry, give the links mass and inertia, configure the joints and actuators, and then actually watch the robot move.

I also found the assignment a bit frustrating at times. There were a lot of settings and steps, and getting the joints and motion inputs configured correctly took some work. But I actually enjoyed figuring it out, and it was satisfying when the simulation finally ran. Measuring the EE position and seeing how closely it matched the forward-kinematics calculations helped me understand how the different parts of the model fit together. Overall, I thoroughly enjoyed this exercise and would like to work with this software more.